You said:

FINAL_SYNOPSIS (... PDF
DOC-20261019-W... PDF
README(2).md Document
Readme (1).md Document
explaination(1).txt Document
README (2).md Document
Phase-1(1).md Document
Phase-2(1).md Document
Phase-3(1).md Document
Phase-4(1).md Document
Phase-5(1).md Document

Analysis all the files.
Give me questions with answers for oral exam of this project.

Note : Jenkins is replaced with GitHub Actions.

Worked for 2m 24s

ChatGPT said:

Cloud Cost Prediction and Optimization — Complete Oral Exam Preparation

I reviewed the supplied project synopsis, Phase-I report, both README files, project explanation, merged-launcher documentation, and Phase 1–5 implementation notes. The questions below are based on those materials and are organized to help you prepare for your project viva.

Important update: Jenkins has been replaced by **GitHub Actions** in your project explanation. Whenever you discuss CI/CD, pipelines, automated testing, or deployment, explain GitHub Actions rather than Jenkins.

1. Understand your project first



Project title: Cloud Cost Prediction and Optimization

Project type: Multi-cloud FinOps platform

Main purpose: Collect cloud billing data, forecast future expenses, detect unusual spending, identify potentially wasteful resources, and support safe cost-optimization actions.

The five main functions

1. Data ingestion

Collect and normalize billing and resource data.

2. Cost forecasting

Estimate future expenses using Prophet or a linear fallback.

3. Anomaly detection

Flag unusually high daily costs.

4. Waste detection

Find idle, stopped, or unattached resources for review.

5. Controlled remediation

Stop or delete eligible resources after explicit approval.

The core project supports Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP). Its main implementation uses Python, FastAPI, PostgreSQL, SQLAlchemy, Prophet, Docker, Terraform, and a browser dashboard. README (2).md Phase-3.md Phase-4.md

2. Your 60-second project introduction

Practice saying this aloud in your own words.

Writing

Good morning, respected examiners.

Our project is titled **Cloud Cost Prediction and Optimization**. It is a multi-cloud FinOps platform designed to help organizations monitor and optimize their cloud expenditure across AWS, Microsoft Azure, and Google Cloud Platform.

The main problem is that organizations may have difficulty comparing bills across different cloud providers, predicting future spending, and identifying resources that continue to incur charges

unnecessarily.

Our application collects cloud billing data through provider-specific adapters, converts it into a common format, and stores it in PostgreSQL. It uses Prophet for time-series forecasting when sufficient historical data is available and a linear-trend fallback otherwise. It also detects unusual spending using statistical thresholds.

A dashboard presents costs, forecasts, anomalies, and potentially idle resources. Resource-changing operations require explicit user approval and are recorded in an audit log.

We use FastAPI for backend APIs, Docker for the local development environment, Terraform for optional sandbox infrastructure, and GitHub Actions for CI/CD automation.

The main objective is to improve cost visibility, support informed decisions, and reduce unnecessary cloud expenditure while maintaining safety and accountability.

Part A — Basic project viva questions

Q1. What is your project about?

Answer: Our project is a cloud cost management platform that collects billing data from AWS, Azure, and GCP, predicts future spending, detects unusual cost increases, identifies potentially wasteful resources, and supports controlled resource optimization.

Q2. What is the problem statement?

Answer: Cloud billing information is distributed across different providers and services. Organizations may struggle to compare costs, identify unused infrastructure, and recognize spending increases before they become expensive. Our system centralizes the data and provides forecasting, anomaly detection, and remediation capabilities.

Q3. Why did you choose this project?

Answer: Cloud computing is widely used, but uncontrolled resource allocation can increase operational expenses. We chose this project to combine cloud computing, backend development, databases, machine learning, DevOps, and automation into a practical cost-management application.

Q4. What is FinOps?

Answer: FinOps means *Cloud Financial Operations*. It is a practice that combines finance, engineering, and business teams to understand cloud spending, improve cost efficiency, and make informed decisions about cloud resources.

Q5. Who will use your application?

Answer: The intended users include FinOps professionals, DevOps engineers, cloud architects, IT administrators, and development teams that need visibility into cloud costs and resource utilization.

Q6. What are the main objectives?

Answer:

1. Collect billing information from multiple cloud providers.
2. Store the information in a normalized database.
3. Display historical costs and spending trends.
4. Forecast future cloud expenses.
5. Detect unusual spending patterns.
6. Identify idle or potentially wasteful resources.
7. Allow controlled remediation with approval and audit logging.
8. Automate testing and delivery through GitHub Actions.

Q7. What is the novelty of your project?

Answer: Its main contribution is integrating multi-cloud data ingestion, cost forecasting, anomaly detection, resource review, and approval-controlled remediation into one workflow. It also supports sample mode for testing without requiring live cloud credentials.

Q8. What is the difference between cloud monitoring and cloud cost optimization?

Answer: Cloud monitoring generally tracks metrics such as CPU utilization, memory, network traffic, and availability. Cloud cost optimization uses cost and utilization information to identify opportunities to reduce unnecessary expenditure while maintaining required performance and availability.

Q9. What is meant by multi-cloud?

Answer: Multi-cloud means using services from more than one cloud provider. Our project targets AWS, Azure, and GCP and converts their provider-specific data into a common internal representation.

Q10. What are the expected benefits?

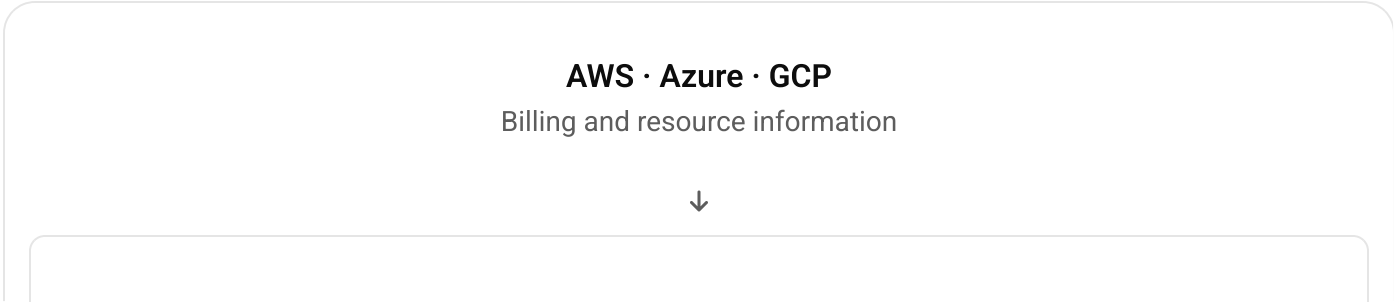
Answer: The system provides centralized cost visibility, forward-looking spending estimates, earlier identification of unusual charges, resource-review suggestions, and an auditable process for approved optimization actions.

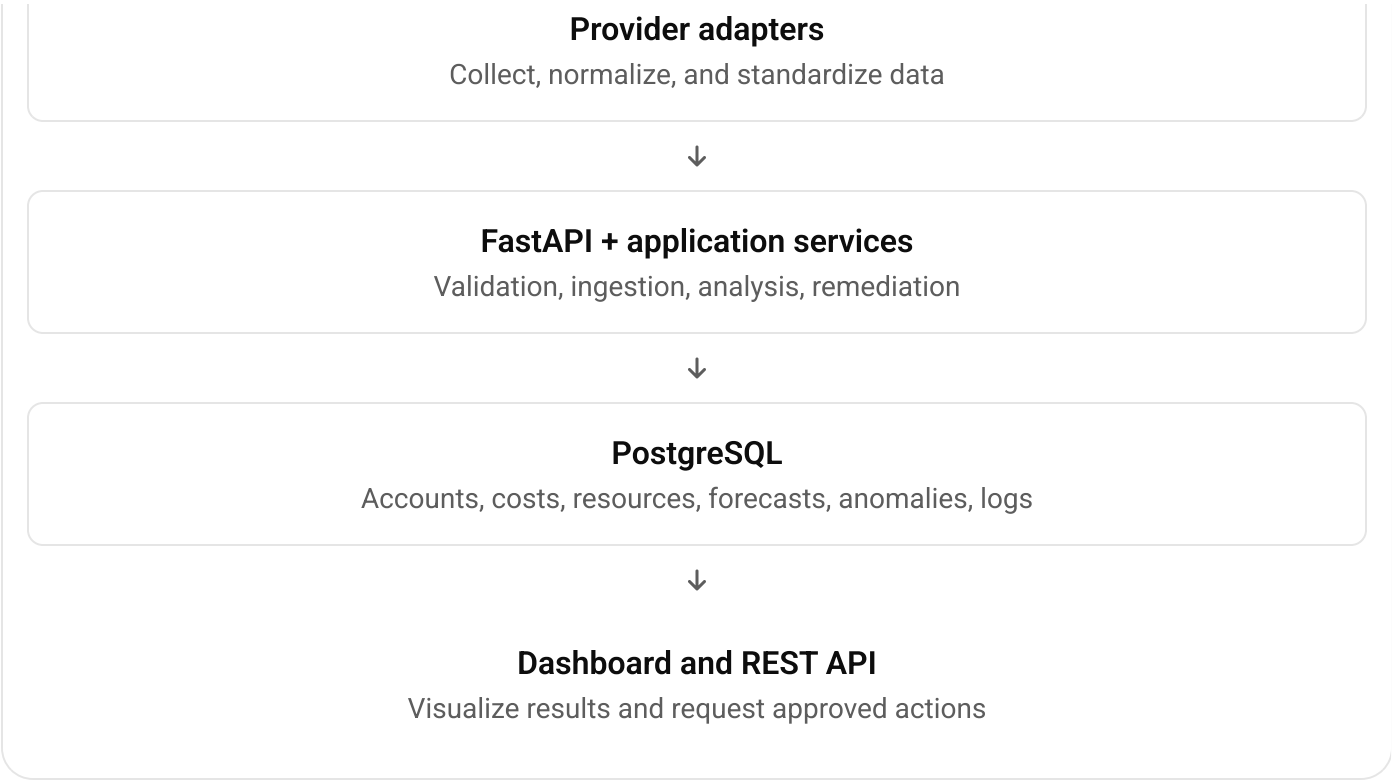
Part B — System architecture and data flow

Q11. Explain the architecture of your project.

Answer: The architecture has five main layers:

1. **Cloud provider layer:** AWS, Azure, and GCP provide billing and resource information.
2. **Adapter layer:** Provider-specific adapters collect data and convert it into common structures.
3. **Application layer:** FastAPI routes validate requests and invoke application services.
4. **Storage and analytics layer:** SQLAlchemy and PostgreSQL store records; analytics services generate forecasts and detect anomalies.
5. **Presentation and remediation layer:** The dashboard presents the results and sends approved resource-action requests to the backend.





The architecture follows the provider-neutral adapter pattern described in the implementation documentation. README (2).md

Q12. What is the complete data flow?

Answer: The user selects providers and a date range. The ingestion service calls the corresponding adapters, receives normalized cost and resource records, and stores them in PostgreSQL. The analytics service prepares daily time series, generates forecasts, and detects anomalies. The dashboard retrieves and displays the results. If a resource requires action, the user approves a supported operation, and the backend executes and logs it.

Q13. Why do you use an adapter design pattern?

Answer: AWS, Azure, and GCP have different APIs, authentication mechanisms, data structures, and resource terminology. The adapter pattern hides those differences behind a common interface, making the application easier to extend and maintain.

Q14. What is a cloud adapter?

Answer: A cloud adapter is a component that communicates with a particular provider and converts its data into the application's common format. The documented interface includes methods such as `collect_costs()`, `find_idle_resources()`, and `remediate()`.

Q15. What is the purpose of FastAPI?

Answer: FastAPI provides the REST API through which the dashboard and other clients can request ingestion, analysis, resource listings, and remediation. It also supports request validation and automatically generated OpenAPI documentation.

Q16. What is a REST API?

Answer: A REST API allows applications to communicate over HTTP using methods such as GET and POST. For example, GET retrieves cost records, while POST can initiate an ingestion or analysis operation.

Q17. What is Uvicorn?

Answer: Uvicorn is an ASGI server that runs the FastAPI application and handles HTTP requests asynchronously.

Q18. What is Pydantic used for?

Answer: Pydantic validates incoming request data and structures API responses using defined schemas and types. It helps reject invalid input before it reaches application logic.

Q19. What is SQLAlchemy?

Answer: SQLAlchemy is the Python ORM and database-access layer. It allows the application to work with database tables through Python models and queries instead of manually writing every SQL statement.

Q20. What is Alembic?

Answer: Alembic manages database schema migrations. It records controlled changes to the database structure, such as creating the forecast and anomaly tables when new functionality is introduced.

Part C — Database and storage

Q21. Why did you choose PostgreSQL?

Answer: PostgreSQL is a relational database suitable for structured billing records, account information, resource inventories, forecasts, and audit logs. It supports transactions, constraints, indexing, and reliable persistent storage.

Q22. Explain the important database tables.

Answer: The core database contains the following tables:

Table	Purpose
`cloud_accounts`	Stores provider account, subscription, or project identities
`cost_records`	Stores normalized billing observations
`cloud_resources`	Stores resource identity, status, type, region, and metadata
`cost_forecasts`	Stores predicted costs and forecast bounds
`cost_anomalies`	Stores unusual spending observations and their severity
`remediation_logs`	Stores requested actions, outcomes, and errors

These tables allow the system to connect historical costs and resource information with the analysis and actions performed on them. explaination.txt

Q23. What is normalization in your project?

Answer: Normalization here means converting provider-specific billing responses into a consistent internal data structure containing fields such as date, service, region, amount, currency, and account identity.

It does not necessarily mean that we are performing every formal relational database normalization step.

Q24. How do you prevent duplicate billing records?

Answer: During ingestion, the service replaces existing cost records only within the selected account and date range. It also upserts resource records using the cloud account and external resource ID. This makes repeated ingestion runs less likely to create duplicate records.

Q25. What is an ORM?

Answer: An Object-Relational Mapper connects application objects to relational database tables. SQLAlchemy is the ORM used in our project.

Q26. What happens if the database schema needs to change?

Answer: We create an Alembic migration describing the schema change, review it, and apply it to the database. In CI/CD, migration validation and execution should be controlled steps rather than unreviewed production changes.

Q27. Why do you store forecasts and anomalies separately from raw costs?

Answer: It separates original billing observations from derived analytical results. Forecasts and anomaly results can be recalculated without modifying the original cost records.

Part D — Machine learning and forecasting

Q28. What is cost forecasting?

Answer: Cost forecasting is the process of estimating future cloud expenditure from historical cost observations. Our application prepares daily cost series and predicts future values for each relevant account, service, region, and currency combination.

Q29. Which forecasting algorithm do you use?

Answer: We use Facebook Prophet, now commonly called Prophet, when its dependency is available, the model is enabled, and sufficient historical data exists. Otherwise, the application uses a linear-trend fallback.

The project notes describe a default seven-day forecast horizon in the analytics implementation, while the original synopsis proposes a 30-day projection. These should not be presented as the same configuration. Phase-3.md FINAL_SYNOPSIS (1).pdf

Q30. What is Prophet?

Answer: Prophet is a time-series forecasting library that models patterns such as trend and seasonality. It is useful for business time series that may exhibit recurring patterns, such as weekly variations in cloud usage.

Q31. Why did you choose Prophet?

Answer: Cloud expenditure can vary with workload trends and recurring weekly activity. Prophet provides a way to model such patterns and returns predicted values along with lower and upper uncertainty bounds.

Q32. What is time-series data?

Answer: Time-series data consists of observations recorded over time in chronological order. In our project, an example is the daily cost of a particular cloud service over several weeks.

Q33. How do you preprocess the billing data?

Answer: The analytics service:

1. Groups records by cloud account, service, region, and currency.
2. Sums records belonging to the same day and group.
3. Sorts observations by date.
4. Fills missing dates between the first and last observation with zero-valued records.
5. Prepares the resulting series for forecasting and anomaly detection.

Q34. Why do you fill missing dates?

Answer: Time-series analysis expects a consistent timeline. Filling missing dates creates a continuous daily series and prevents gaps in the input from disrupting the forecasting calculations.

Important viva point: A missing billing record does not always mean the actual cost was zero. In a production system, we should distinguish a genuine zero-cost day from delayed, incomplete, or missing billing data.

Q35. What is a linear-trend fallback?

Answer: It is an alternative forecasting method based on a fitted linear trend. It allows the application to produce an estimate when Prophet is unavailable, insufficient historical data exists for Prophet, or Prophet cannot fit the series.

Q36. What is the difference between Prophet and linear regression?

Answer: Linear regression models a relationship using a fitted line. Prophet can model a trend together with seasonal patterns and uncertainty intervals. A linear-trend fallback is simpler and useful when the more sophisticated forecasting model cannot be applied.

Q37. What are forecast confidence or uncertainty bounds?

Answer: The lower and upper bounds indicate a range of plausible forecast values according to the model's uncertainty estimates. They are not guarantees that the actual future cost will fall inside that range.

Q38. How do you know whether your forecasting model is accurate?

Answer: We should compare predictions against actual costs over historical periods that were not used to fit the model. Metrics such as Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) can quantify prediction errors. The supplied report does not provide a validated numerical forecast-accuracy score.

Q39. Does your project train a deep learning model?

Answer: No deep learning model is specified in the core implementation notes. The documented forecasting approach uses Prophet and a linear-trend fallback.

Q40. Why do you use pandas?

Answer: Pandas is used to organize and preprocess the historical cost observations into daily time-series data suitable for forecasting and statistical analysis.

Part E — Anomaly detection

Q41. What is anomaly detection?

Answer: Anomaly detection identifies observations that differ significantly from expected behavior. In our project, it flags daily cloud costs that exceed a calculated upper threshold.

Q42. How does your anomaly detection algorithm work?

Answer: The detector calculates the expected cost using preceding daily observations, calculates an upper threshold, and compares the actual daily cost with that threshold. If actual cost exceeds the threshold, the system stores an anomaly with its severity and supporting details.

The documented threshold is:

$$U = \max(\mu + 1.96\sigma,\ 1.25\mu,\ 0.01)$$

Where:

- U = upper threshold.
- μ = mean of preceding daily costs.
- σ = standard deviation of preceding daily costs.
- 0.01 = minimum threshold in the relevant currency units.

This formula is applied to the relevant historical cost series. Phase-3.md

Q43. What does the value 1.96 mean?

Answer: The factor 1.96 is commonly associated with a two-sided 95% interval under a normal-distribution assumption. In our formula, it scales the historical standard deviation to create a statistical margin above the mean. The complete anomaly rule is a practical threshold, not a guarantee of a 95% false-alarm property.

Q44. How do you classify anomaly severity?

Answer: According to the documented implementation:

Actual cost divided by upper threshold	Severity
At least 2.0	Critical

At least 1.5 but below 2.0	High
Greater than 1.0 but below 1.5	Medium

Q45. Why does the detector use only preceding costs?

Answer: It avoids including the observation being tested in its own baseline. Otherwise, an unusually high value could increase its own expected cost and threshold, making the anomaly harder to detect.

Q46. What is the difference between forecasting and anomaly detection?

Answer: Forecasting estimates what may happen in the future. Anomaly detection identifies actual observations that are unusually high relative to a calculated baseline.

Q47. Can your anomaly detector produce false positives?

Answer: Yes. A legitimate workload increase can trigger an alert, and a slow but persistent increase might not cross the threshold. The results should therefore be investigated alongside workload changes, deployments, service usage, and billing information.

Q48. What happens after an anomaly is detected?

Answer: The application stores the anomalous observation, expected cost, threshold, severity, explanation, and related calculation metadata. The results can then be retrieved through the analysis API and displayed on the dashboard.

Part F — Waste detection and remediation

Q49. What is a wasteful cloud resource?

Answer: It is a resource that may incur unnecessary costs because it is unused, underutilized, unattached, or no longer needed. Examples include unattached storage volumes and idle virtual machines.

Q50. How does your project identify waste?

Answer: It discovers candidate resources through provider adapters and applies the supported resource-state and eligibility checks. The project documents examples such as stopped EC2 instances, unattached EBS volumes, deallocated Azure VMs, stopped GCP VMs, and unattached disks.

The separate CloudOptima README also describes a heuristic for EC2 instances with average CPU utilization below 5% over 14 days. That threshold belongs to the documented CloudOptima application and should not automatically be attributed to every resource detector in the core Phase 1–5 implementation. Readme (1).md README (2).md

Q51. What is remediation?

Answer: Remediation is the process of taking an action to address an identified problem. In our project, it means requesting a supported operation, such as stopping or deleting an eligible cloud resource, after the required approval.

Q52. Does your application automatically delete resources as soon as they are detected?

Answer: No. The documented workflow requires explicit approval. The service validates the requested action and resource eligibility before attempting the operation.

Q53. Why is approval required?

Answer: A resource that appears idle might still be needed for a scheduled task, disaster recovery, or an infrequent production workload. Approval helps prevent accidental downtime and data loss.

Q54. What is an audit log?

Answer: An audit log is a record of actions and their outcomes. Our `remediation_logs` table stores information such as the requested action, requester, status, provider response or error, and timestamp.

Q55. What happens if a remediation operation fails?

Answer: The application records the failed operation and its error details in the audit log. This helps the user investigate issues such as insufficient permissions, invalid resource identifiers, or provider API failures.

Q56. What is the difference between stopping and deleting a resource?

Answer: Stopping attempts to halt a resource while retaining it for possible future use. Deleting removes the resource and can be irreversible. Stopping does not necessarily eliminate all charges, because storage, reservations, or other associated services may continue to incur costs.

Q57. What is least privilege?

Answer: The principle of least privilege means giving an identity only the permissions required to perform its specific tasks. For example, cost ingestion should use read permissions, while remediation should use separately controlled permissions for only the supported resource actions.

Q58. How does sample mode help?

Answer: Sample mode supplies deterministic example records so we can test ingestion, analytics, dashboard rendering, and simulated remediation without connecting to live cloud accounts. Sample remediation does not change real infrastructure.

Part G — GitHub Actions, DevOps and CI/CD

This section is especially important because your project uses GitHub Actions instead of Jenkins.

Q59. What is CI/CD?

Answer: CI/CD stands for Continuous Integration and Continuous Delivery or Continuous Deployment. It automates activities such as testing, building, validating, and delivering software.

- **Continuous Integration (CI):** Automatically checks changes when developers push code or open pull requests.
- **Continuous Delivery:** Keeps the software ready for release, with production deployment generally requiring approval.
- **Continuous Deployment:** Automatically releases validated changes to production when all configured conditions pass.

Q60. Why did you replace Jenkins with GitHub Actions?

Answer: GitHub Actions integrates directly with GitHub repositories, pull requests, branches, and releases. It allows us to define automated workflows using YAML files in the `.github/workflows/` directory.

It can run tests, build Docker images, validate Terraform configurations, and perform controlled deployment steps without maintaining a separate Jenkins server for a basic workflow.

Q61. What is GitHub Actions?

Answer: GitHub Actions is GitHub's workflow automation platform. A workflow can run when an event occurs, such as a code push, pull request, or manually triggered release.

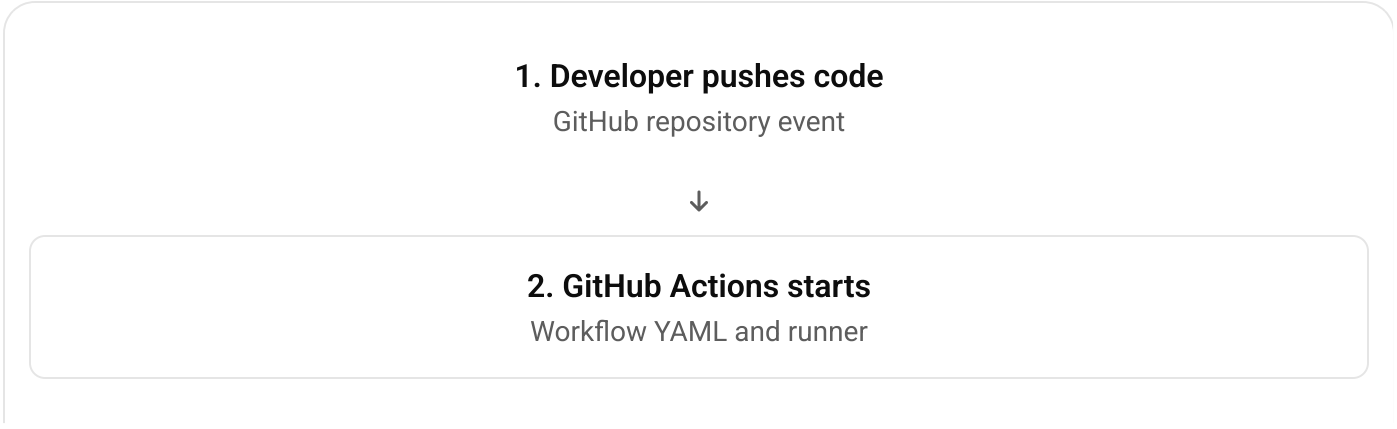
Q62. What are the main components of GitHub Actions?

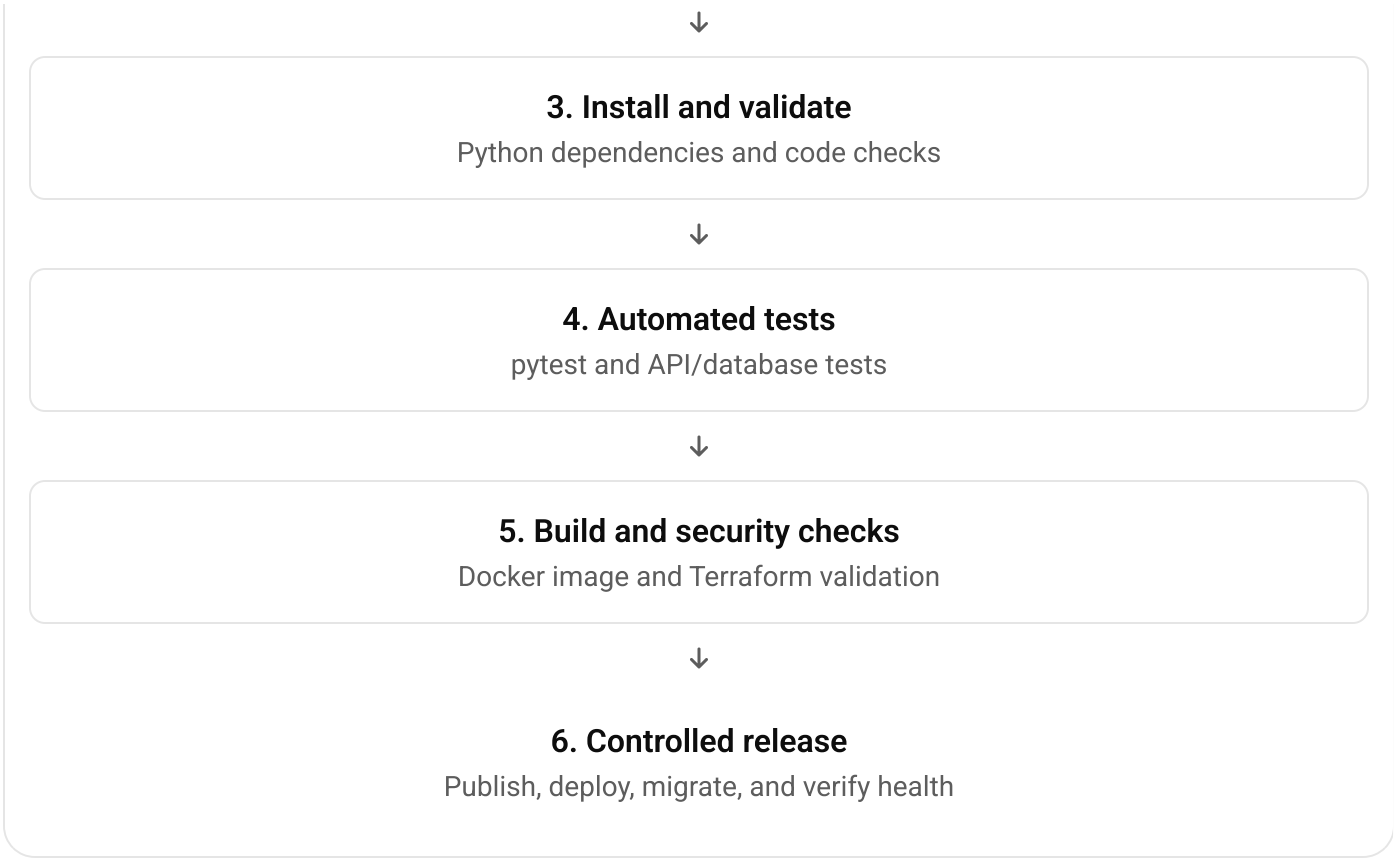
Answer:

- **Workflow:** The complete automation process defined in a YAML file.
- **Event or trigger:** The event that starts a workflow.
- **Job:** A group of steps executed on a runner.
- **Runner:** The machine that executes the job.
- **Step:** An individual command or reusable action.
- **Action:** A reusable unit of automation.

Q63. How will GitHub Actions work in your project?

Answer: When code is pushed or a pull request is opened, GitHub Actions can check out the repository, install Python dependencies, run pytest, build the Docker image, and validate the Terraform configuration. If the checks pass, a separately controlled release workflow can publish the image and deploy the application.





This is the recommended pipeline for the project, based on the supplied explanation document. Do not claim every stage is already running unless you have verified the actual workflow files and execution history. explainationtxt

Q64. Where do you define a GitHub Actions workflow?

Answer: In a YAML file inside the repository's `.github/workflows/` directory, for example `.github/workflows/ci.yml`.

Q65. What is the difference between Jenkins and GitHub Actions?

Answer: Jenkins is a general-purpose automation server that commonly requires separate server or agent management. GitHub Actions is integrated into GitHub and uses workflow files and runners to automate repository events. Both can perform CI/CD tasks; we use GitHub Actions for this project.

Q66. What tests should the pipeline run?

Answer: The pipeline should run tests covering the health endpoint, adapter registry, sample ingestion, date-range behavior, cost aggregation, missing-date handling, forecasting fallback, anomaly detection, and remediation validation. It should also validate database migrations and

API behavior where configured.

Q67. What happens if a test fails?

Answer: The workflow reports the failure and the affected job fails. Later dependent jobs, such as image publishing or deployment, should not run unless the workflow explicitly permits them to continue.

Q68. How do you store secrets in GitHub Actions?

Answer: Sensitive credentials should be stored in GitHub Actions secrets or an appropriate external secret manager, not hard-coded in source files or workflow YAML. Where supported, short-lived identity federation is preferable to long-lived cloud keys.

Q69. Should Terraform automatically create production infrastructure after every push?

Answer: No. Terraform validation and planning can be automated, but production changes should have appropriate review, restricted permissions, and approval gates. Our sandbox configuration also disables providers and compute creation by default.

Q70. Is GitHub Actions the same as Docker?

Answer: No. GitHub Actions automates the workflow, whereas Docker packages an application and its dependencies into a container image. GitHub Actions can run Docker commands to build and test that image.

Part H — Docker and Terraform

Q71. What is Docker?

Answer: Docker is a containerization platform that packages software and its dependencies into containers. It helps the application run consistently across supported development and deployment environments.

Q72. What is Docker Compose?

Answer: Docker Compose defines and manages multiple related containers through a configuration file. In our project, it is used to run the FastAPI application and PostgreSQL database together for local development.

Q73. What is the difference between an image and a container?

Answer: An image is a packaged template containing the application and its dependencies. A container is a running instance created from that image.

Q74. Why do you use Terraform?

Answer: Terraform is an Infrastructure as Code (IaC) tool. It describes infrastructure declaratively, allowing us to define repeatable sandbox networking and optional compute resources for AWS, Azure, and GCP.

Q75. What is Infrastructure as Code?

Answer: Infrastructure as Code means defining and managing infrastructure through configuration files rather than creating every resource manually. It improves repeatability, reviewability, and consistency.

Q76. Explain the main Terraform commands.

Answer:

Command	Purpose
`terraform init`	Initializes the working directory and providers
`terraform fmt -check`	Checks configuration formatting
`terraform validate`	Checks configuration structure and consistency
`terraform plan`	Previews proposed infrastructure changes
`terraform apply`	Applies approved changes
`terraform destroy`	Removes resources managed by the selected state

Viva tip: `terraform validate` does not prove that a deployment will succeed or that permissions and cloud quotas are sufficient. Always review the plan before applying changes.

Q77. Why are compute resources disabled by default in Terraform?

Answer: Creating virtual machines can incur charges and introduce security risks. Disabled-by-default settings reduce accidental resource creation and allow the team to validate the configuration before intentionally enabling infrastructure.

Q78. Is Terraform required to run the local dashboard?

Answer: No. Terraform is optional for the local application. The API and database can be run using Docker Compose without provisioning new cloud infrastructure.

Part I — API endpoints and practical demonstration

Q79. What are the important endpoints in your core application?

Answer: The main endpoints documented across the phases are:

Method	Endpoint	Purpose
GET	`/api/v1/health`	Check API health
POST	`/api/v1/ingestion/sync`	Ingest or simulate cloud cost data
GET	`/api/v1/ingestion/accounts`	List connected account records
GET	`/api/v1/ingestion/costs`	Retrieve cost records
GET	`/api/v1/ingestion/resources`	Retrieve discovered resources
POST	`/api/v1/analysis/run`	Run forecasting and anomaly detection
GET	`/api/v1/analysis/forecasts`	Retrieve forecasts
GET	`/api/v1/analysis/anomalies`	Retrieve anomalies
GET	`/api/v1/remediation/resources`	List resources available for review
POST	`/api/v1/remediation/resources/{resource_id}`	Request a supported resource action
GET	`/api/v1/remediation/logs`	Retrieve remediation audit logs

Phase-3.md

Phase-4.md

Q80. How can you test the API without building a separate frontend?

Answer: We can use the automatically generated Swagger UI at `http://localhost:8000/docs`. It allows us to inspect endpoints, enter parameters, submit requests, and view responses.

Q81. How would you demonstrate your project during the viva?

Answer: Follow this sequence:

1. Start the application and PostgreSQL using Docker Compose.
2. Open Swagger UI and demonstrate the health endpoint.
3. Use sample mode to ingest data from the supported providers.
4. Retrieve the stored accounts, costs, and resource candidates.
5. Run the analytics endpoint and show the forecast and anomaly results.
6. Open the dashboard and explain the cost trends and resource list.
7. Demonstrate approval-controlled remediation in sample mode.
8. Show the audit log and, if available, the GitHub Actions workflow and its test results.

For the main Phase 1–5 application, the supplied Phase 5 notes document a browser dashboard served by FastAPI at `/`. The README files also describe a separate CloudOptima dashboard and a launcher that exposes both applications. Make sure you demonstrate the correct application for the version you are presenting. Phase-4.md README.md

Q82. How do you know ingestion was successful?

Answer: Check the provider-specific result returned by the ingestion endpoint, the number of cost and resource records collected, and the records stored in PostgreSQL. A successful HTTP response alone does not necessarily mean every cloud provider succeeded.

Q83. What happens if AWS succeeds but Azure fails?

Answer: The ingestion service reports status, counts, and errors separately for each provider. A failure from one provider should not hide successful results from another.

Q84. Why might the dashboard not display a forecast?

Answer: The forecast may not have been generated yet, insufficient history may be available, the analytics endpoint may have failed, or the forecast records may not have been stored. The documented workflow requires running analysis after ingestion.

Q85. How can you distinguish sample results from live cloud results?

Answer: Check the selected adapter and ingestion configuration, inspect the provider-level results, and verify whether actual cloud credentials and API calls were used. Sample mode is explicitly intended to demonstrate the application without modifying live infrastructure.

Part J — Security, testing and troubleshooting

Q86. How do you protect cloud credentials?

Answer: Credentials should be supplied through provider-supported credential chains, environment configuration, or a secret manager. They should never be committed to GitHub. The application should use separate read and remediation permissions, with the smallest practical scope.

Q87. Does the current project provide complete production-grade security?

Answer: No. The project includes important safeguards such as approval checks, action validation, least-privilege design, and audit logging. However, the supplied explanation notes that a production deployment should add appropriate authentication and authorization, security review, and stronger operational controls.

Q88. What is the difference between authentication and authorization?

Answer: Authentication establishes who the user or service is. Authorization determines which operations that identity is permitted to perform.

Q89. What is the purpose of pytest?

Answer: Pytest is the Python testing framework used to write and execute automated tests for application functionality, including API behavior, adapter contracts, ingestion, analytics, and remediation validation.

Q90. What is an integration test?

Answer: An integration test verifies that multiple components work together. For example, an ingestion integration test might verify that an API request invokes the ingestion service and persists normalized records in the database.

Q91. What is the difference between unit testing and integration testing?

Answer: Unit testing checks an individual function or component in isolation. Integration testing checks interactions between components such as the API, database, adapters, and services.

Q92. What does the health endpoint do?

Answer: The `/api/v1/health` endpoint checks that the API responds and returns its configured status and environment. A successful response does not necessarily prove that all dependencies, such as PostgreSQL or cloud APIs, are healthy.

Q93. What does `WinError 10061` mean in your project notes?

Answer: It generally indicates that a connection was refused. The project notes associate this dashboard error with the local API not running or failing to start. We should check the API logs, database health, and configured port.

Q94. What would you do if PostgreSQL does not start?

Answer: Check the Docker Compose logs, confirm that port 5432 is available, verify the database configuration and environment variables, and ensure the database container passes its health check before starting dependent application operations.

Q95. What would you do if cloud ingestion fails?

Answer: Check the selected provider, credentials, account or subscription identifiers, required API permissions, billing configuration, network connectivity, provider API limits, and the error returned by the adapter. I would also verify that the failure is not being confused with sample-mode behavior.

Q96. What would you do if GitHub Actions fails?

Answer: Open the failed workflow run, identify the failing job and step, inspect its logs, reproduce the issue locally where possible, correct the configuration or code, and push the fix. I would not bypass a failing test simply to make deployment succeed.

Q97. What is a database migration failure?

Answer: It occurs when a schema change cannot be applied successfully. The cause may be an invalid migration, incompatible schema state, missing permissions, or database connectivity problems. The migration history and database state should be checked before attempting recovery.

Q98. What are the limitations of the current project?

Answer:

- Forecast quality depends on the quantity and quality of historical billing data.
- Statistical anomaly thresholds may generate false positives or miss some patterns.
- Resource-state checks do not always prove that a resource is genuinely unnecessary.
- Live integrations depend on valid credentials, permissions, and provider-specific configuration.
- Remediation can cause downtime or data loss if used incorrectly.
- The supplied report does not provide validated numerical cost savings or forecast-accuracy results.
- The application needs additional security, load testing, and operational validation before production deployment.

DOC-20261019-WA0003.pdf

Part K — Project phases, report questions and future scope

Q99. Explain Phase 1.

Answer: Phase 1 established the foundation: FastAPI, a health endpoint, PostgreSQL models, SQLAlchemy, Alembic migrations, Docker configuration, a common cloud adapter interface, and basic tests. It deliberately did not perform live cloud ingestion or resource-changing actions.

Q100. Explain Phase 2.

Answer: Phase 2 introduced optional Terraform modules for AWS, Azure, and GCP sandbox networking and optional compute resources. Providers and compute creation are disabled by default. The notes state that Terraform validation was completed, but no cloud plan or apply was run during that phase.

Q101. Explain Phase 3.

Answer: Phase 3 introduced the cloud adapters, normalized cost and resource records, sample-data support, PostgreSQL persistence, and ingestion API endpoints. It is designed to be read-only with respect to cloud resources.

Q102. Explain Phase 4.

Answer: Phase 4 introduced daily cost preprocessing, Prophet forecasting, the linear-trend fallback, statistical anomaly detection, forecast and anomaly database tables, and analysis API endpoints.

Q103. Explain Phase 5.

Answer: Phase 5 introduced the browser dashboard, resource listing, approval-controlled stop/delete operations, provider response persistence, and success/failure audit logging.

Q104. What is the future scope of your project?

Answer: Future improvements include:

1. Scheduled ingestion and automated analysis.
2. Configurable budgets and alert notifications.
3. Forecast evaluation using MAE, RMSE, and historical backtesting.
4. Better anomaly detection for service-specific patterns.
5. Role-based access control and more detailed approval history.
6. Dry-run previews before resource-changing actions.
7. Better billing-data quality checks, including currency differences and delayed records.
8. More comprehensive provider integration and security testing.
9. Production deployment, monitoring, backups, and recovery procedures.

Q105. How does your project contribute to sustainability?

Answer: By helping users identify potentially unnecessary cloud resources, the project can support more responsible use of computing infrastructure. This relates to SDG 12, Responsible Consumption and Production; SDG 13, Climate Action; and SDG 9, Industry, Innovation and Infrastructure.

However, the application does not directly measure carbon emissions, and actual environmental benefits would need to be evaluated rather than assumed.

Q106. What is the business impact of your project?

Answer: It can improve visibility into cloud expenditure, help teams investigate unexpected charges, and support informed resource optimization. Actual savings depend on the resources identified, the decisions made, and the costs associated with the resulting actions.

Q107. Why is your project feasible?

Answer: It uses established technologies and open-source libraries, supports local development with Docker Compose, and includes sample data for demonstrations. Terraform is optional, so a live cloud environment is not required for every development or testing task.

Q108. What did you learn from this project?

Answer: The project combines backend API design, relational database modeling, cloud SDK integration, time-series analytics, anomaly detection, containerization, infrastructure automation, CI/CD, testing, and secure resource-management workflows.

Part L — Tricky questions examiners may ask

These questions test whether you understand the implementation rather than only memorizing definitions.

Q109. Your synopsis says 30-day forecasting, but your Phase 4 notes say seven days. Which is correct?

Answer: The original synopsis proposes a 30-day forward projection, while the documented Phase 4 implementation uses a configurable forecast horizon with seven days as the default. I would explain that the horizon depends on the implementation configuration and should be set to 30

days if that is the required final project behavior.

Q110. Your synopsis mentions Streamlit, but Phase 5 describes HTML, CSS and JavaScript. Why?

Answer: The supplied documents describe different interface versions or project components. The core Phase 5 implementation documents a browser dashboard served by FastAPI, while the synopsis and the separate CloudOptima application describe Streamlit-based interfaces. I would demonstrate the interface belonging to the final application version and ensure the report matches it.

Q111. Is your project completely automated?

Answer: Data ingestion, analytics, and CI/CD tasks can be automated. However, the documented resource-remediation workflow requires explicit approval before changing a cloud resource. This is intentional because stopping or deleting resources can affect production workloads.

Q112. Can you guarantee that the forecast is correct?

Answer: No. Forecasts are estimates based on historical observations and model assumptions. Their reliability must be measured using representative historical data and appropriate evaluation metrics.

Q113. Does detecting a stopped virtual machine mean it is wasting money?

Answer: Not necessarily. A stopped virtual machine may still incur storage, reserved-capacity, or other associated charges, but it may also be intentionally stopped. The application identifies candidates for review rather than proving that every candidate should be deleted.

Q114. What happens when Prophet is not installed?

Answer: The documented analytics service can use the linear-trend fallback when Prophet is unavailable or cannot fit the series. This keeps the local application usable under supported fallback conditions.

Q115. Why not delete every resource with low utilization?

Answer: Low utilization alone does not prove that a resource is unnecessary. The resource could support a critical but infrequent task. Resource ownership, workload requirements, environment tags, and approval must be considered before remediation.

Q116. What is the most important safety feature?

Answer: The combination of explicit approval, action validation, resource eligibility checks, restricted cloud permissions, and audit logging. These controls reduce the risk of unintended infrastructure changes and make actions traceable.

Q117. Why use GitHub Actions instead of running tests manually?

Answer: GitHub Actions makes testing repeatable and automatically runs configured checks when repository events occur. It helps detect errors early and prevents later delivery stages from proceeding when required checks fail.

Q118. Has the project achieved measurable cost savings?

Answer: The supplied project report does not provide validated numerical savings or forecasting-accuracy results. The project implements a workflow intended to support cost optimization, but actual savings must be measured against a suitable baseline after evaluating real usage and approved actions.

Part M — Last-minute revision sheet

Memorize these essential facts before your viva.

Cloud providers

AWS, Microsoft Azure, GCP

Backend and database

Python, FastAPI, Uvicorn, Pydantic, SQLAlchemy, PostgreSQL, Alembic

Analytics

Prophet, pandas, linear-trend fallback, rolling upper-bound anomaly detection

DevOps

Docker, Docker Compose, Terraform and GitHub Actions — not Jenkins

Safety

Explicit approval, eligibility checks, least privilege, audit logs, sample mode

Important URLs

Dashboard: `http://localhost:8000/`

API documentation: `http://localhost:8000/docs`

Health: `http://localhost:8000/api/v1/health`

The 10 questions to prioritize if your viva is tomorrow

1 **Explain your project in one minute.**

2 **Why do you use the adapter pattern?**

3 **Why PostgreSQL?**

4 **How does Prophet work in your project?**

5 **How do you detect anomalies?**

6 **What is the role of FastAPI?**

7 **Why Docker and Terraform?**

8 **Why GitHub Actions instead of Jenkins?**

9 **How do you prevent dangerous remediation?**

10 What are the limitations?

Final advice for your viva

- Learn the **project introduction, architecture, database tables, forecasting algorithm, anomaly formula, and GitHub Actions workflow** especially well.
- Be prepared to explain what happens when ingestion, forecasting, database access, or remediation fails.
- Do not claim that the model achieves a particular accuracy or that the system saves a particular percentage of money without experimental evidence.
- Know the difference between features proposed in the synopsis and features documented in the implementation.
- If asked to explain a function or endpoint, describe its input, processing, output, and safety checks rather than only naming the technology.

The most important point: demonstrate that you understand how billing data travels from the cloud adapters through PostgreSQL and the analytics service to the dashboard, and how GitHub Actions helps validate changes without bypassing the project's safety controls.